RAS 585: Machine Learning and Artificial Intelligence (Fall 2026)

Instructor: Shenghan Guo

Assignment 2

This assignment is worth 200 points. It will be due on Wednesday, 11/4/2026, and submitted through Canvas. Code should be written in Python. Other programming languages may be used with the instructor's approval. A typed report (".docx", ".pdf") should be submitted as a separate document along with the code component (".py", ".ipynb") through Canvas.

Data: A dataset, "iris.data", is provided to you for Part 2; a description of the variables can be found in "iris.names". Please read the description file before starting Part 2. Part 1 and Part 3 use images loaded or provided as described in those parts — see below.

Note: You may consider the samples as independent and identically distributed (iid). For Part 2, the input attributes are "sepal length", "sepal width", "petal length", and "petal width" (all in cm, one observation per row). The final column of "iris.data" is the ground-truth class label, with three classes: Iris-setosa, Iris-versicolor, and Iris-virginica.

Coding component: Your coding component for this assignment should include two parts: (1) Code that you write in a conventional manner without AI assistance. You can use online Python tutorials or coding forums to confirm syntax. No AI is allowed in this part. (2) Code generated by an AI agent. You can choose Codex, Claude, or Gemini. After obtaining the AI-generated code, you will compare your handwritten code with the AI-generated code and then add comments ("#…") in the Python script/notebook generated by AI. Note that these comments should highlight the logical and functional differences between the two versions of code.

Grading: The point value for each part and question is explicitly listed in this assignment. You must make sure that you follow the data, report, and coding guidelines. Failure to submit the required report or coding component will result in a loss of credit. Specifically, missing AI-generated code will result in a 30% deduction, missing handwritten code will result in a 30% deduction, and a missing report will result in a 40% deduction.

Part 1 (60 pts)

Use the built-in color image from the scikit-image library for this part. No file download is required:

"from skimage import data"
"img = data.coffee()"

This is a 400×600 RGB image of a cup of coffee with rich color and texture variation, which makes it a good test case for clustering-based compression.

1. **Image compression with hierarchical clustering**: Agglomerative hierarchical clustering can be used for image compression. Divide the image into non-overlapping c × c windows; each window becomes a c^2-dimensional vector ($c^2 \times 3$ for an RGB image — flatten each window with "ndarray.flatten('F')" in Python). Treat these window-vectors as your samples and perform agglomerative hierarchical clustering with single linkage for a given number of clusters k. The cluster centers (reference vectors) and the cluster index for each window are what would be sent over a communication line; at the receiving end, the image is reconstructed by looking up each window's reference vector from its index. Write a program that does this for $k \in \{6, 10, 14\}$ and $c \in \{20, 50, 80\}$ (9 combinations total; use "sklearn.cluster"). For each (k, c) pair:

(a) save the compressed image as ".jpg" or ".png"; (b) calculate the *reconstruction error* (the sum of squared differences between the original and reconstructed pixel values); and (c) calculate the *compression rate* — the ratio of the compressed representation's size (in bits) to the original image's size (in bits), computed explicitly as follows:

compression rate = (compressed size in bits) ÷ (original size in bits)
compressed size = codebook size + index size
codebook size = $k \times c^2 \times 3 \times 8$ bits (k reference vectors, each of dimension $c^2 \times 3$, stored at 8 bits per value)
index size = $N \times \text{ceil}(\log_2 k)$ bits (N = (image height ÷ c) × (image width ÷ c) is the number of non-overlapping c × c windows; $\text{ceil}(\log_2 k)$ is the number of bits needed to store one index into the k-vector codebook)
original size = image height × image width × 3 × 8 bits (8 bits per color channel per pixel)

(Hint: use nested "for" or "while" loops to exhaust all (k, c) combinations, and keep pixel values as integers — i.e., "int" in Python — before visualizing the compressed images.) (28 pts)

2. Based on the reconstruction errors and compression rates from Question 1, identify the best combination of k and c, and justify your choice: is the pair with the lowest reconstruction error always the best overall choice once compression rate is also considered? (6 pts)

3. **Image compression with k-means clustering**: Repeat the image-compression task from Question 1 using *k-means clustering* instead of hierarchical clustering, with the same $k \in \{6, 10, 14\}$ and $c \in \{20, 50, 80\}$. For each (k, c) pair, save the compressed image and calculate both the reconstruction error and the compression rate as in Question 1. (Hint: swap in "sklearn.cluster.KMeans".) (18 pts)

4. Based on the results of Question 3, identify the best combination of k and c. Then compare the hierarchical-clustering and k-means results overall: which method gives a better reconstruction-error / compression-rate trade-off for this image, and why might that be the case? (8 pts)

Part 2 (100 pts)

Use "iris.data" for this part (see the Data and Note sections above for the input attributes and class labels).

1. For the class Iris-setosa only (n = 50 samples), estimate the 4-dimensional multivariate *kernel density* using a *Gaussian kernel* with bandwidth (smoothing parameter) $h \in \{1.5, 0.5, 0.1\}$. This is the multivariate generalization of the Gaussian kernel density estimator covered in the Nonparametric Methods chapter. For each value of h: build the 4-dimensional kernel density estimate from the 50 setosa samples, then visualize it as a function of one input attribute at a time by evaluating the estimate along a grid spanning that attribute's observed range while holding the other three attributes fixed at their sample means. This gives 4 plots per value of h (one per attribute), for 12 plots total. Briefly comment on how the choice of h affects the smoothness of the estimated density. (15 pts)

2. Split the iris data into a training set (50% of the samples in each class) and a testing set (the remaining 50% of the samples in each class), so that the class proportions are preserved in both sets. This is called *stratified sampling*. (10 pts)

3. Perform classification with *k-nearest neighbors* for $k \in \{3, 7, 11, 15\}$. For each k, train a k-nearest-neighbors classifier on the training set from Question 2 (scikit-learn's "neighbors" module) and evaluate it on the testing set. Report the *confusion matrix* and prediction accuracy for each k, identify the k giving the best performance, and briefly comment on the result. (25 pts)

4. Using the training and testing sets from Question 2, perform classification with a *classification tree* using the *Gini index* as the impurity measure (scikit-learn's "tree" module). Report the confusion matrix and prediction accuracy. (20 pts)

5. Using the training and testing sets from Question 2, perform classification with *Linear Discriminant Analysis* (not Fisher's Linear Discriminant; scikit-learn's "LinearDiscriminantAnalysis"). Report the confusion matrix and prediction accuracy. (20 pts)

6. Among k-nearest neighbors, the classification tree, and LDA, which achieves the best classification accuracy on your testing set? Explain why you think that method outperforms the others, based on what you know about each method's assumptions and decision mechanism. (5 pts)

7. Summarize the differences in the classification mechanism of k-nearest neighbors, the classification tree, and LDA. (5 pts)

Part 3 (40 pts)

Use "Assignment2_YOLO_image.jpg" for this part (provided to you — an image containing several common, everyday objects such as people, vehicles, or animals).

1. **Object recognition with neural nets**: Convolutional neural networks (CNNs) are powerful tools in computer vision. Several model families built on CNNs can recognize objects in an image; one widely used example is You-Only-Look-Once (*YOLO*). Pretrained YOLO models are accessible through Python libraries (e.g., the "ultralytics" package). Apply a pretrained YOLO model to the provided image to recognize the objects it contains. Comment on which objects were correctly and incorrectly recognized. (35 pts)

2. Briefly explain the connection and difference between CNNs and a multilayer perceptron. (5 pts)